Can a Model Tell Cause from Correlation? A Simple Pipeline on the Corr2Cause Dataset

Chaitanya Sai Meka, Rakshita, Dhanvin Vadlamudi, Kasula Lalithendra, Jagruthi Pulumati
CSAI Project, Phase 1 (Machine Learning)
Code: https://github.com/Jag2007/AI_ML_Project

***Abstract*—"Two things move together" does not mean "one causes the other". The Corr2Cause dataset tests exactly this. Each example gives a short text listing which variables are related, and a claim such as "A directly causes B". The task is to say whether the claim is definitely true (1) or not (0). We solved it in five steps, one notebook per team member: clean the data, turn the text into a cause-and-effect graph, build features, train classifiers, and finally draw the predicted graph and ask it "what if" questions. Using only the words of the text, our model scores an F1 of 0.47. Adding the graph-based features raises this to 0.88. By reading the dataset generator's code we matched its exact claim definitions, so the graph step alone (no learning) reproduces 98.5% of the test labels. Our final model, a Random Forest, scores an F1 of 0.957 on the unseen test set, close to the 0.952 of the no-learning rule. 89% of the cause arrows it draws are correct, and our graphs answer 92 of 96 "what if" questions correctly, against 0 of the 32 real effects for an empty or a reversed graph.**

I. Introduction

The problem. Ice cream sales and drowning cases both go up in summer. They are *correlated*, but ice cream does not cause drowning; hot weather causes both. Large language models often make this mistake and read correlation as causation.

The dataset. Corr2Cause [1] turns this into a yes/no task. Here is a real example (shortened):

> *Premise:* There is a closed system of 3 variables, A, B and C. A correlates with B. A is independent of C. B is independent of C.
> *Hypothesis:* There is a common effect (collider) of A and C.
> *Label:* 0 (not valid)

The *premise* lists which pairs of variables are related ("correlate") and which are not ("independent"). The *hypothesis* is a claim about cause and effect. The label is 1 only if the claim is true in *every* cause-and-effect picture that fits the premise. If even one fitting picture breaks the claim, the label is 0.

Our goal. Instead of a black box, we wanted a pipeline where every prediction can be traced back to a graph we can draw and explain. So we combined a classic causal method (the PC algorithm) with simple machine learning models.

II. Related Work

We group earlier work into four lines and close with the gap that our project fills (Table I).

Causal graphs and discovery from independences. Pearl [2], [3] gave the theory of Bayesian networks and of reasoning with causal graphs, and d-separation [4] turns "independent given Z" into a rule that can be read off a graph. Verma and Pearl [5] showed that two DAGs imply the same independences exactly when they share the same skeleton and the same colliders, so data can only identify a *Markov equivalence class* (MEC), drawn as a partly directed graph [6]. The PC algorithm [7] recovers this class from independence tests, and Meek's rules [8] orient every edge that can be oriented. PC is consistent even with many variables [9]. Score-based search such as GES [10] and continuous optimisation such as NOTEARS [11] are alternatives, but they need numeric data, while Corr2Cause only gives the independences as text.

Causal relation extraction from text. Early systems found cause–effect pairs with hand-written lexical and syntactic patterns [12], [13]. SemEval-2010 Task 8 [14] made cause–effect one of nine relation classes between noun pairs, and BECauSE 2.0 [15] and the Causal News Corpus [16] annotate causal language in running text. These works find *what the text claims* about causes. They do not check whether a claim *follows* from statistical evidence, and they do not produce a graph that can answer intervention questions.

Causal reasoning benchmarks for language models. Corr2Cause [1] asks whether a causal claim follows from a list of correlations and independences. Its authors found that 17 large language models score close to chance. Fine-tuned BERT-style models score highly on the original test set, but their scores drop sharply when the hypotheses are paraphrased or the variable names are changed, so they learn surface patterns rather than the reasoning. CLadder [17] tests all three rungs of Pearl's ladder with text questions built from causal graphs, and Kıcıman et al. [18] and Zečević et al. [19] argue that LLMs repeat causal facts they have seen rather than reason causally. Corr2Cause is framed as natural language inference, the same premise–hypothesis format as MNLI [20].

Neuro-symbolic AI. Neuro-symbolic systems let a learned component read the input and a symbolic component do exact reasoning [21]. The project brief asks for this split: text extraction proposes relations, and the graph stage enforces acyclicity, quantifies uncertainty and answers interventions.

Feature engineering and models. We use TF-IDF [22] for text, filter-based feature selection [23] with chi-squared and mutual information [24], logistic regression, Random Forest [25] and gradient boosting [26] from scikit-learn [27], the bootstrap [28] for confidence intervals, and pgmpy [29] for PC and Bayesian networks.

TABLE I
EARLIER WORK AND THE GAP WE ADDRESS

Work	What it does	What it lacks for our task
PC, GES, NOTEARS [7], [10], [11]	Learn a graph from numeric data	Cannot read text
Patterns, SemEval-8, BECauSE [12], [14], [15]	Extract causal pairs from text	No check that a claim follows; no graph or interventions
Corr2Cause LLMs and fine-tuned models [1]	Classify premise–hypothesis pairs end to end	Near chance (LLMs) or brittle to rewording (fine-tuned)
CLadder [17]	Text questions on all rungs of causation	Evaluation only, no extraction pipeline
Ours	Regex parsing → PC → features → classifier → DAG and Bayesian network	Every prediction traces back to a graph

III. THEORETICAL BACKGROUND

Bayesian networks. A DAG G over variables $V_1, \dots, V_n$ with parents $\mathrm{Pa}(V_i)$ and a probability table at each node defines

$$P(v_1, \dots, v_n) = \prod_{i=1}^{n} P\big(v_i \mid \mathrm{pa}(v_i)\big). \tag{1}$$

Under the causal Markov and faithfulness assumptions, the independences in the data are exactly the d-separations in the graph:

$$X \perp\!\!\!\perp Y \mid Z \ \text{ in } P \iff X \perp_G Y \mid Z. \tag{2}$$

A path is blocked by Z if it has a non-collider in Z, or a collider $X \to W \leftarrow Y$ such that neither W nor any descendant of W is in Z.

Equivalence class and labels. The Markov equivalence class $\mathcal{M}$ of a premise is the set of DAGs whose d-separations match the premise, which are the DAGs with the same skeleton and colliders [5]. For a hypothesis h and a DAG G, let $h(G) \in \{0, 1\}$ say whether the claim holds in G. Corr2Cause labels a hypothesis valid only if it holds in every member, and we measure how close it is to that with

$$y(h) = \prod_{G \in \mathcal{M}} h(G), \qquad \text{mec_support}(h) = \frac{1}{|\mathcal{M}|} \sum_{G \in \mathcal{M}} h(G). \tag{3}$$

PC algorithm. PC starts from the complete undirected graph. It removes the edge $X - Y$ if some set S of neighbours makes them independent, $X \perp\!\!\!\perp Y \mid S$, and stores S as their separating set. For every non-adjacent pair X, Y with a common neighbour $Z \notin S_{XY}$, it orients the collider $X \to Z \leftarrow Y$. Meek's rules then orient every remaining edge whose direction is forced. The output represents exactly the class $\mathcal{M}$.

Interventions. Forcing $X = x$, written $do(X = x)$, removes the arrows into X. Dropping the factor of X from Eq. (1) gives the truncated factorisation

$$P\big(v \mid do(X{=}x)\big) = \prod_{i:\, V_i \neq X} P\big(v_i \mid \mathrm{pa}(v_i)\big)\Big|_{X=x}, \tag{4}$$

and when $\mathrm{Pa}(X)$ blocks every back-door path, the back-door adjustment gives $P(y \mid do(x)) = \sum_z P(y \mid x, z)\, P(z)$ with $Z = \mathrm{Pa}(X)$ [3]. We report the average causal effect

$$\mathrm{ACE}(X \to Y) = P(Y{=}1 \mid do(X{=}1)) - P(Y{=}1 \mid do(X{=}0)). \tag{5}$$

Text features. With n documents, term frequency tf and document frequency df, sublinear TF-IDF is

$$w_{t,d} = \big(1 + \ln \mathrm{tf}_{t,d}\big)\Big(\ln \frac{1+n}{1+\mathrm{df}_t} + 1\Big), \tag{6}$$

and each document vector is scaled to unit length. Mutual information $I(F;Y) = \sum_{f,y} p(f,y) \log \frac{p(f,y)}{p(f)\,p(y)}$ ranks features by how much they reduce uncertainty about the label, including non-linear relations.

Classifiers. Logistic regression fits $p_i = \sigma(\beta^\top x_i + \beta_0)$ by minimising the class-weighted log-loss with an L_2 penalty,

$$\min_{\beta} \ \frac{\|\beta\|^2}{2} - C \sum_i w_{y_i} \big[y_i \ln p_i + (1 - y_i) \ln(1 - p_i)\big], \tag{7}$$

with class weights $w_c = n/(2\, n_c)$, so the rare valid class counts as much as the common one. Random Forest averages deep trees grown on bootstrap samples, each split chosen to reduce the Gini impurity $1 - \sum_k p_k^2$. Gradient boosting builds $F_m = F_{m-1} + \nu\, h_m$, where each small tree h_m fits the negative gradient of the log-loss and ν is the learning rate.

Evaluation. With precision P and recall R, $F_1 = 2PR/(P+R)$. Rows that share a premise are not independent, so confidence intervals use a cluster bootstrap: resample whole premises with replacement, recompute F_1, and take the 2.5th and 97.5th percentiles.

IV. STEP 1: CLEANING THE DATA (NOTEBOOK 01)

We used the official train, validation and test splits from Hugging Face (Table II). Each row has the text (`input`), the `label`, the number of variables, and a `template` that says which type of claim it is.

Checking the data. We looked at the first and last rows, the shape and the column types. We counted missing values and duplicate rows. There were none, so nothing had to be filled in or removed.

Cleaning. Every text has one "Hypothesis:" marker, so we split it into two columns: `premise` and `hypothesis`. We also removed extra spaces. The same premise is reused with up to 90 different claims, so we gave each premise an ID number.

Removing leakage. A few premises appeared in two splits at once (2 in train and validation, 3 in validation and test). If the model sees a premise during training and again in testing, its score looks better than it really is. We left the test set

TABLE II
NUMBER OF ROWS BEFORE AND AFTER CLEANING

Split	Before	After	Premises
Train (sampled)	205,734	35,340	400
Validation	1,076	1,026	19
Test	1,162	1,162	21

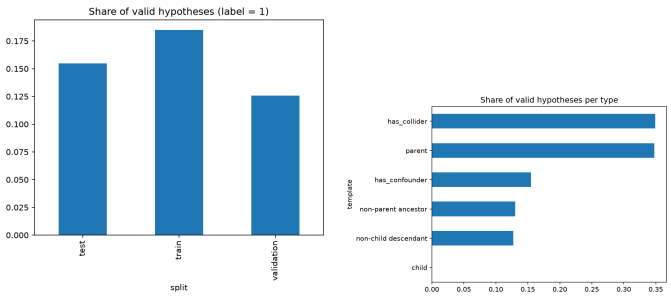


Fig. 1. Left: only 13–19% of claims are valid in each split. Right: share of valid claims for each claim type in train. "Child" claims are never valid.

as it was and removed these shared premises from train and validation.

Making train smaller. Train has about 206,000 rows, which is slow to work with. We randomly picked 400 premises (seed 42) and kept all of their claims, giving 35,340 rows. The share of valid claims stayed the same (18.5%).

V. LOOKING AT THE DATA (EDA)

Few valid claims. Only 13–19% of claims are valid (Fig. 1, left). A lazy model that always answers "not valid" would already be over 80% accurate. So accuracy is misleading here. Instead we use *precision* (of the claims we called valid, how many really were), *recall* (of the truly valid claims, how many we found) and *F1* (one number that balances the two). We also told the models to give more weight to the rare valid class.

A hidden shortcut. "Child" claims are never valid (Fig. 1, right), yet their text ("X directly causes Y") looks exactly like "parent" claims. So the `template` column gives away the answer. We did not use it as a feature; we only used it later to study mistakes.

Harder test set. Train only has graphs with 5–6 variables, but validation and test also have graphs with 2–4 variables. So the test set includes some cases the model never saw in training.

VI. STEP 2: TEXT TO GRAPH (NOTEBOOK 02)

Reading the text. The sentences follow fixed patterns, so simple regular expressions are enough. From each claim we pick out the two variables (single capital letters) and the claim type (direct cause, indirect cause, common effect, common cause) using keywords. We also added safety checks: a variable must be one of A–F and must appear in the premise, a variable cannot cause itself (A→A), and unknown wording is rejected. All real rows passed, and broken examples we made by hand were caught.

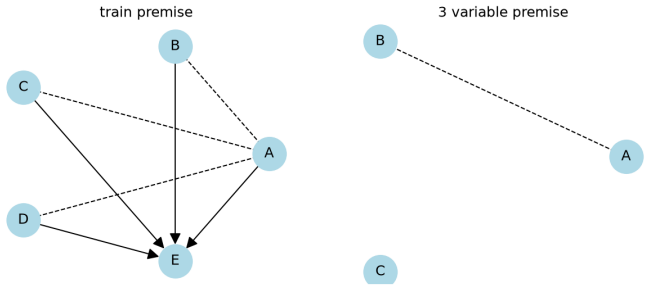


Fig. 2. PC output for two premises. Solid arrows have a known direction; dashed lines could point either way. Left: E is a common effect of A–D. Right: only A and B are linked, and C stands alone.

TABLE III
CLAIM DEFINITIONS USED BY THE CORR2CAUSE GENERATOR

Claim	A path from X to Y
X directly causes Y	X→Y
X causes sth. else which causes Y	X→ ··· →Y (2+ arrows)
Collider (common effect)	X→ ··· →Z← ··· ←Y
Confounder (common cause)	X← ··· ←Z→Y

Running PC. PC normally needs numeric data to test for independence. Here the premise already tells us which pairs are independent, so we let pgmpy's PC read those facts directly (`ci_test="independence_match"`). PC then (1) starts with all pairs linked and removes links between independent pairs, (2) finds patterns like X→Z←Y, where Z is a common effect, and (3) applies its fixed rules to set more directions. Links it cannot decide are left undirected (Fig. 2).

Counting fitting graphs. For each undirected link we try both directions. We keep every version that has no loops and fits the same independence facts. This gives us all the DAGs in the equivalence class, between 1 and 144 per premise. For each claim we then compute mec_support, the share of these DAGs where the claim is true (Eq. 3). If it is 1, the claim holds in every fitting graph, so it should be valid.

What does each claim mean? Our first definitions were the textbook ones: a collider is a shared direct child, a confounder a shared direct parent, and "X causes something else which causes Y" means Y is a descendant of X but not a direct child. With these, the mec_support rule matched only 93.7% of train labels. So we read the code that generated the labels [30]. It walks every path between X and Y and names the path by its arrows (Table III). Two differences matter. First, "X causes something else which causes Y" only needs a path of two or more arrows, even if X→Y also exists. Second, colliders and confounders can be reached through longer paths, and a loop in the generator that counts the wrong way means a confounder only counts when the common cause is directly linked to the second variable.

With these definitions the same no-learning rule matches 96.6% of train, 97.5% of validation and 98.5% of test labels. The largest group of remaining mismatches (41%) are "parent" claims where every fitting DAG has the arrow X→Y but the

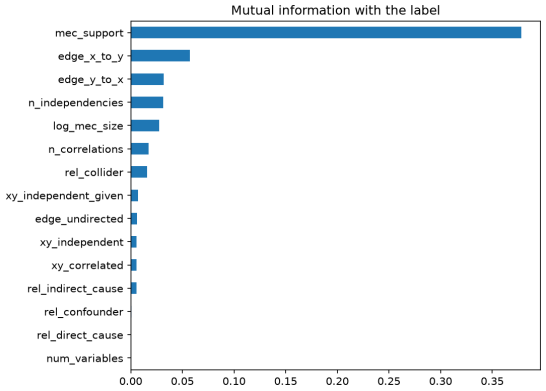


Fig. 3. How much each graph feature tells us about the label (train). mec_support, which comes from PC, is clearly the most useful.

label is 0. We treat these as label noise; a public audit of the test split also reports wrong labels in both directions. We also noticed that the letters follow the causal order: across all 440 premises, PC never points an arrow from a later letter to an earlier one. This is why "child" claims, which always name the later letter as the cause, are never valid.

VII. Step 3: Building Features (Notebook 03)

A model cannot read text or graphs directly, so we turn each claim into a row of numbers called features.

Graph features (15). The claim type (one column per type); how many correlations and independences the premise lists; whether X and Y are said to correlate, to be independent, or to be independent once other variables are known; whether PC drew X→Y, Y→X or an undirected X–Y; the number of fitting DAGs (on a log scale); and mec_support. We rescaled these to a similar range, using only train data to set the scale.

Text features. TF-IDF gives each word or word pair a score: high if it appears often in this text but rarely in others. We made one TF-IDF table for the premise (135 terms) and one for the claim (106 terms), learned from train only. We kept one-letter words because A–F are the variable names.

Which features matter? We scored each feature by how much it tells us about the label (mutual information, Fig. 3). The 0/1 and count features were treated as discrete and the other two as continuous. mec_support is far ahead of everything else, followed by the arrow-direction features. The top text words are things like "collider" and variable letters, which say nothing about the specific pair being asked about. Keeping only the top 80 text terms changed F1 by about 0.02 (0.88 vs 0.90), which is within the noise of a 19-premise validation set, so we kept all of them.

Text vs graph features. Table IV trains the same simple model (logistic regression) on three feature sets. With text only, the model calls almost everything valid: it finds 95% of valid claims, but only 31% of its "valid" answers are right. The graph features fix this, and using both is best among the learned models. The no-learning rule still scores higher (0.91): with balanced class weights the linear model trades precision for recall, and it cannot copy the sharp step at mec_support = 1. The tree models in Step 4 learn that step.

TABLE IV
Which features help? (logistic regression, validation set)

Features	Precision	Recall	F1
Text only	0.311	0.953	0.469
Graph only	0.840	0.853	0.846
Text + graph	0.884	0.884	**0.884**
Rule mec_support = 1 (no learning)	0.846	0.977	0.906

TABLE V
Results (validation F1; test precision, recall and F1)

Model	Val F1	Test P	Test R	Test F1
Logistic Regression	0.878	0.922	0.983	0.952
Random Forest (chosen)	**0.929**	0.927	0.989	**0.957**
Gradient Boosting	0.879	0.927	0.989	0.957
mec_support rule only (no ML)	0.906	0.922	0.983	0.952

Two different models agree (Fig. 4): the graph features, not the words, drive the predictions.

VIII. Step 4: Training and Testing Models (Notebook 04)

Setup. Each claim becomes 256 numbers (text + graph features). We tried three models: logistic regression, Random Forest (300 decision trees voting together) and gradient boosting (small trees added one after another, each fixing the last one's mistakes). For each model we tried a few settings with `GridSearchCV` and 5-fold cross-validation.

One detail matters here: we used `GroupKFold`, which keeps all claims of the same premise in the same fold. Otherwise the model could see a premise in training and be checked on the same premise, which would make the score look too good. We picked the model with the best validation F1, and used the test set only once, at the very end.

Results. Random Forest had the best validation F1, so we chose it (Table V) and used the test set only to report it. It is slightly better than the plain mec_support rule on both validation (0.929 vs 0.906) and test (0.957 vs 0.952), but the test gap is well inside the confidence interval below. The honest reading is that the symbolic step does almost all of the work once the claim definitions are right, and the learned model mainly tidies up the edges of that rule. This is the neuro-symbolic split the brief asks for: the text step proposes, the graph step decides.

How sure are we? The test set has only 21 premises, so the score could change with a different test set. To check, we re-sampled whole premises 1,000 times (the bootstrap) and recomputed F1 each time. The F1 stayed between 0.921 and 0.982 in 95% of the runs.

Where it goes wrong. The model made 16 mistakes out of 1,162 (Fig. 5). Half are false alarms on "parent" claims, the same label disagreements found in Step 2. It made no mistakes on "child" claims. Graphs with 2–4 variables, which

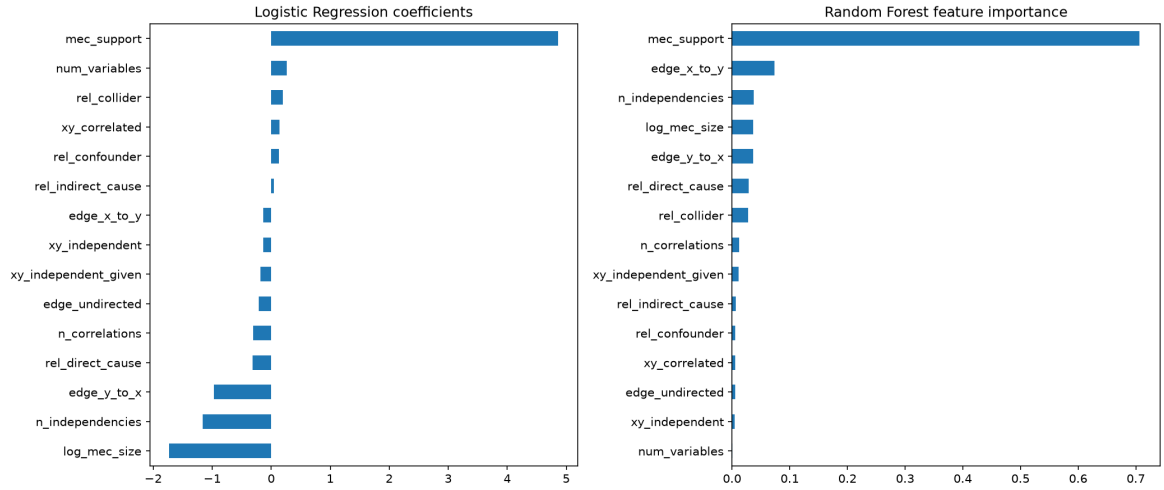


Fig. 4. Left: logistic regression weights. mec_support pushes strongly towards "valid" (+4.9); a large equivalence class, many independences and a Y→X arrow push towards "not valid". Right: Random Forest importance, where mec_support alone takes 0.71 of the total.

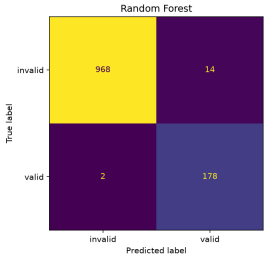


Fig. 5. Random Forest on test: 968 correct "not valid", 178 correct "valid", 14 false alarms and 2 missed valid claims.

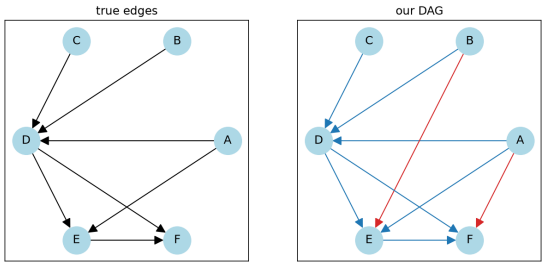


Fig. 6. One test premise. Left: the true arrows. Right: our predicted DAG. Blue arrows are correct; the two red arrows (B→E, A→F) are extra ones that PC supports but the dataset marks as not valid.

never appeared in training, had 1 mistake in 126 rows (0.8%), against 1.4% for 5–6 variable graphs. This is because the graph features come from the parser and PC, not from memorised sentences.

IX. Step 5: Drawing the Graph and Asking "What If" (Notebook 05)

Building the DAG. For each test premise, every "X directly causes Y" claim that the model calls valid becomes an arrow. We add arrows starting with the ones the model is most sure about. An arrow is skipped if it points from a variable to itself, uses a variable not in the premise, or would create a loop. All 21 test graphs came out as proper DAGs. Out of 73 arrows drawn, 65 are correct (**precision 0.89**), and no true arrow was missed (recall 1.00). All 8 wrong arrows are also drawn by PC, so they come from the same label disagreement as before (Fig. 6).

Bayesian network and "what if" questions. A Bayesian network is a DAG with probabilities attached to each node. It lets us ask "what happens to Y if we *force* X to a value?", written $do(X)$. This is different from just observing X, and it is the real test of cause and effect.

Corr2Cause has no numbers, only text, so we made some. We gave the true graph random probabilities, generated 5,000 samples from it, and fitted our predicted DAG on those samples with pgmpy. Then we measured the effect of X on Y with the ACE of Eq. (5). For the chain A→B→C, the true effect of A on C is 0.490 and our correctly drawn graph gives 0.498. If the arrows are reversed, the graph still fits the same data but gives 0.000, and wrongly says that C affects A.

Intervention queries. We used the five test premises whose graph is fully known (one DAG in the class). Each premise gets its own random probabilities, and we ask about every ordered pair of variables in both directions: 96 queries, 32 of which have a real effect. A query counts as correct only if the graph gets both whether there is an effect at all (a directed path) and its size within 0.05. The first condition stops a tiny true effect from "matching" a graph with no arrow. Table VI compares our DAGs with two baselines fitted on the same data.

Our DAGs answer 28 of the 32 real effects correctly; both baselines answer none. All four misses come from one premise where the model left out C→D and C→E, two arrows that PC says are certain but the dataset labels as not valid. So getting the arrows right, in the right direction, is what makes the "what if" answers right.

TABLE VI
INTERVENTION QUERIES ON THE FIVE FULLY KNOWN TEST GRAPHS

Graph	All queries	Real effects	Mean |error|
Our predicted DAG	**92 / 96**	**28 / 32**	**0.011**
Same DAG, arrows reversed	36 / 96	0 / 32	0.126
Empty graph	64 / 96	0 / 32	0.064

X. Conclusion and Next Steps

Turning the text into a graph with a parser and the PC algorithm, using the claim definitions of the dataset's own generator, explains 97–98% of the labels with no learning. Adding these graph features to simple text features and training a Random Forest gives a test F1 of 0.957 (95% CI 0.921–0.982), with no data leaking between splits. Text features alone reach only 0.47, which supports the brief's point that fluent text features cannot replace formal causal reasoning. Most remaining mistakes come from a small set of labels that disagree with PC.

Limitations. Corr2Cause is generated from templates, so our regex parser would not work on paraphrased or real text, and the letters follow the causal order, which a model could exploit. Train only has 5–6 variable premises. The Bayesian network data is simulated from the PC graph, not measured.

Phase 2. We plan to replace the regex parser and TF-IDF with a fine-tuned transformer that reads paraphrased text, keep PC for the reasoning, and test both on paraphrased and renamed-variable versions of the data, where the Corr2Cause authors found fine-tuned models break down.

Contribution Matrix

Each person owned one notebook (✓ = owner).

Task	Chaitanya	Rakshita	Dhanvin	Lalithendra	Jagruthi
Loading and checking data	✓				
Cleaning, leakage, sampling	✓				
EDA and plots	✓				
Text parser and safety checks		✓			
PC algorithm, mec_support		✓			
Building features			✓		
Feature selection, text vs graph			✓		
Model tuning and selection				✓	
Evaluation, error bars, mistakes				✓	
Building the DAG, arrow precision					✓
Bayesian network, "what if"					✓
Report and repository					✓

References

[1] Z. Jin, J. Liu, Z. Lyu, S. Poff, M. Sachan, R. Mihalcea, M. Diab, and B. Schölkopf, "Can large language models infer causation from correlation?" in *Proc. ICLR*, 2024.

[2] J. Pearl, *Probabilistic Reasoning in Intelligent Systems: Networks of Plausible Inference*. Morgan Kaufmann, 1988.

[3] J. Pearl, *Causality: Models, Reasoning, and Inference*, 2nd ed. Cambridge Univ. Press, 2009.

[4] D. Geiger, T. Verma, and J. Pearl, "Identifying independence in Bayesian networks," *Networks*, vol. 20, no. 5, pp. 507–534, 1990.

[5] T. Verma and J. Pearl, "Equivalence and synthesis of causal models," in *Proc. UAI*, 1990, pp. 255–268.

[6] S. A. Andersson, D. Madigan, and M. D. Perlman, "A characterization of Markov equivalence classes for acyclic digraphs," *Ann. Statist.*, vol. 25, no. 2, pp. 505–541, 1997.

[7] P. Spirtes, C. Glymour, and R. Scheines, *Causation, Prediction, and Search*, 2nd ed. MIT Press, 2000.

[8] C. Meek, "Causal inference and causal explanation with background knowledge," in *Proc. UAI*, 1995, pp. 403–410.

[9] M. Kalisch and P. Bühlmann, "Estimating high-dimensional directed acyclic graphs with the PC-algorithm," *J. Mach. Learn. Res.*, vol. 8, pp. 613–636, 2007.

[10] D. M. Chickering, "Optimal structure identification with greedy search," *J. Mach. Learn. Res.*, vol. 3, pp. 507–554, 2002.

[11] X. Zheng, B. Aragam, P. Ravikumar, and E. P. Xing, "DAGs with NO TEARS: Continuous optimization for structure learning," in *Proc. NeurIPS*, 2018.

[12] C. S. G. Khoo, S. Chan, and Y. Niu, "Extracting causal knowledge from a medical database using graphical patterns," in *Proc. ACL*, 2000, pp. 336–343.

[13] R. Girju, "Automatic detection of causal relations for question answering," in *Proc. ACL Workshop on Multilingual Summarization and Question Answering*, 2003, pp. 76–83.

[14] I. Hendrickx *et al.*, "SemEval-2010 Task 8: Multi-way classification of semantic relations between pairs of nominals," in *Proc. SemEval*, 2010, pp. 33–38.

[15] J. Dunietz, L. Levin, and J. Carbonell, "The BECauSE corpus 2.0: Annotating causality and overlapping relations," in *Proc. Linguistic Annotation Workshop*, 2017, pp. 95–104.

[16] F. A. Tan *et al.*, "The Causal News Corpus: Annotating causal relations in event sentences from news," in *Proc. LREC*, 2022.

[17] Z. Jin *et al.*, "CLadder: Assessing causal reasoning in language models," in *Proc. NeurIPS*, 2023.

[18] E. Kıcıman, R. Ness, A. Sharma, and C. Tan, "Causal reasoning and large language models: Opening a new frontier for causality," *arXiv:2305.00050*, 2023.

[19] M. Zečević, M. Willig, D. S. Dhami, and K. Kersting, "Causal parrots: Large language models may talk causality but are not causal," *Trans. Mach. Learn. Res.*, 2023.

[20] A. Williams, N. Nangia, and S. Bowman, "A broad-coverage challenge corpus for sentence understanding through inference," in *Proc. NAACL*, 2018, pp. 1112–1122.

[21] A. d'Avila Garcez and L. C. Lamb, "Neurosymbolic AI: The 3rd wave," *Artif. Intell. Rev.*, vol. 56, pp. 12387–12406, 2023.

[22] G. Salton and C. Buckley, "Term-weighting approaches in automatic text retrieval," *Inf. Process. Manage.*, vol. 24, no. 5, pp. 513–523, 1988.

[23] I. Guyon and A. Elisseeff, "An introduction to variable and feature selection," *J. Mach. Learn. Res.*, vol. 3, pp. 1157–1182, 2003.

[24] A. Kraskov, H. Stögbauer, and P. Grassberger, "Estimating mutual information," *Phys. Rev. E*, vol. 69, no. 6, p. 066138, 2004.

[25] L. Breiman, "Random forests," *Mach. Learn.*, vol. 45, no. 1, pp. 5–32, 2001.

[26] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," *Ann. Statist.*, vol. 29, no. 5, pp. 1189–1232, 2001.

[27] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.

[28] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*. Chapman & Hall, 1993.

[29] A. Ankan and A. Panda, "pgmpy: Probabilistic graphical models using Python," in *Proc. SciPy*, 2015.

[30] Corr2Cause authors, "Data generation code," https://github.com/causalNLP/corr2cause, file `code/data_gen.cpp`, accessed Oct. 2026.